Angelo Kelly

basicCalculator.cpp

Assignment 1.5 (Basic Calculator)

The purpose of this program is to simply create a calculator that can perform simple operations such as addition, subtraction, multiplication, division and modulo operation. At first sight, it doesn't solve any particular issue that hasn't been solved before and in greater ways, however, I intended for this calculator to be more interactive with the user, walking them through the proper structure of an operation. By doing so, it ensures that anybody, from toddler to elder, may use this basic calculator.

Here is a list of all the inputs the program will receive:

- First number
- Mathematical operators
- Second number
- Confirmation to run the program again or exit it

Likewise, here is a list of the outputs the program will produce:

- Indication to write a first number
- Indication to write an operator from the displayed list
- Indication to write a second number
- A display of the result of the calculation done
- Ask the user if they wish to do another calculation
- A series of error messages regarding cases of division by zero or invalid inputs (i.e. letters instead of numbers or operators)

Variables

In order to make this program work as desired, the following variables were used:

Variable	Type	Purpose
num1	double	Holds first number given by user.
operation	char	Holds the operator the user chooses.
num2	double	Holds second number given by user.
result	double	Stores the result of the operation.
running	bool	Allow the while-loop inside the program to repeat itself as long as it is true.

answer	string	Stores the response of the user to whether or not they want to keep using the program.
valid_result	bool	Checks if the result of the operation makes sense for a basic calculator.

Key Design Choices

Here is an explanation of some design choices made in the program, explained from the top of the program to the bottom of it.

"num1", "num2" and "result" as double types: The reason for these three variables to be doubles instead of int or float is simply because a double type variable allows the calculator to work with decimals while also maintaining a high precision.

The bool "running" and string "answer": This two variables are deeply related, for they were made to allow the user to run the program several times without the need to shut it down forcefully. The bool "running" is used as condition for the main while-loop to work, as long as "running" is true, the while-loop will keep on going, and therefore, the program never stops. But, what if the user does not want to keep using the calculator but doesn't want to, or is unable to, shut it down forcefully? For that reason, the program asks the user at the end of each result displayed if they want to do another calculation or not. The user's response is store in "answer", if they say "Y" or "y", then "running" remains true and the while-loop runs once more; however, if the user inputs "N" or "n", the bool "running" is set to false, and since the condition of the main while-loop is false, the program stops, allowing the user to close the program without interacting directly with the terminal or the source code. In addition, "answer" was made as a string instead of a char type of variable to handle the scenario where the user inputs something that is neither "Y" nor "n" nor the lower-case character.

While loops and do-while loops: In several blocks of code it can be noticed the presence of a do-while loop within a common while loop. The reason why I used a while loop as the main loop for the calculator to work is because I wanted the user to be able to run the program as much as they want with no clear limits, but, the condition "running" should've been read beforehand, reason why a do-while was not used as the main loop of the program. Likewise, do-whiles are used to store information in the "operator" and "answer" variables because they have to ask the user at least once what do they want to write, store the information in the variables, and compare it with the conditions to decide whether or not the program can continue.

bool valid_result: This bool variable serves to validate the equation done by the user. Suppose, for instance, that the user is a young person and types “tun-tun-zahur 67 skibbidiRizz” instead of a mathematical operation, perhaps as a joke. For this cases, the program should be able to read the user’s inputs and determine if they are valid or not for the sake of the program. Since obviously nothing in the example resembles a mathematical equation in the slightest, the bool “valid_result” is set as false, provoking the program to not be able to display or show what is stored in result, for there should not be anything in the first place. No valid operation was ever done; therefore, no valid result can ever be shown.

while(!(std::cin>>num1/num2)){}: In addition to the bool “valid_result”, this while loop is also set whenever the user is asked to input either their first or second number for the operation. Before, we established that the bool turns false whenever the user writes something that cannot be considered a mathematical value. How? By checking if the user’s input can be read as a double or not. If yes, then the input is valid and the program keeps running as intended. In the contrary, given the case the user inputs a non-numerical character that cannot be converted to a double, the NOT operator will set the while loop as true, and then will alert the user that letters and/or symbols are not allowed until the user inputs a proper, real number.

std::cin.clear(); and std::cin.ignore(1000, ‘\n’);: I found these two functions of cin very useful to handle the case where the user inputs extra information, such as statements, multiples operators displayed together, etc. As a matter of fact, every time the user writes an invalid input, the program might skip a part of the invalid input, but it might also store the rest of it into another variable, which will eventually cause a bug from the user’s perspective, as they will notice how the program warns them that “no letters or symbols are allowed as numbers. Try again”, but they haven’t even written their second number yet. Moreover, if the user reaches the end, but instead of typing “Y” or “y” writes “YEAH!!”, the program will still read the first character “Y” and, according to the if-statement, will allow the while loop to run once more. However, the remaining characters “EAH!!” will get store in num1, resulting, once again, in a bug. Thus, the necessity for cin.clear() and cin.ignore(). The first one will clean up the entirety of the last invalid input given by the user, while the second one directly ignores up to a thousand characters might be written, including spaces, skips over them.

Switch-case instead of if-else statements: I decided to handle the operations with a switch-case format for purposes of readability and simplicity given the case scenarios and the limited amount of possible user inputs. An if-else statement would’ve work out as well, but a partner coder could find the code as “messy” or “exhausting” after reading if-else more than five times.

division by zero warning: This is a common edge-case to be mindful about, whether the user makes it on purpose or by accident, division by zero is undefined by mathematical definition. Therefore, if the user enters zero as their second number for a division, a warning will be

displayed telling the user that dividing by zero is undefined. Consequentially, "valid_result" is set as false, skipping the line where results are shown and instead asking directly to the user if they want to keep using the calculator.

#include <cmath> and std::fmod(): Finally, the function "fmod" was borrowed from the library of "cmath" because in C++, modulo is not intended to work for doubles, hence why it was easier to include an extra library to handle the case of modulo operators.

Program steps (Algorithm)

1- Ask user for numbers and operator.
2- Check if the inputted values are valid.
3- If invalid, tells the user to type a correct input.
4- If valid, calculates operation and prints result.
5- Ask the user if they want to use the calculator again.

Functions

int main( )

Purpose: It is the entry point of the program, contains the calculator's loop, logic, program output and user's input.

Sample Input/Output

Output	**Input**
Enter a number:	10
Enter an operator:	*
Enter another number:	2
The operation result is: 20	
Please only respond with Y or N. Would you like to do another operation? [Y / N]	Y // y // N // n

Testing

Test case 1: 16/4

Expected result: 4

```
Enter a number: 16
Enter intended operation (ONLY [+] [-] [x] [*] [/] [%]): /
Enter another number: 4
The operation result is: 4
Please only respond with Y or N. Would you like to do another operation? [Y / N]:
```

Test case 2: edge case, invalid input; letters.

Expected result: Error or repeated messages until user enters valid input

```
Enter a number: Thought it was a number, but it was me!! DIO!!
Letters and/or symbols are not recognized as numbers. Please, try again.
Enter a number: 33
Enter intended operation (ONLY [+] [-] [x] [*] [/] [%]): But I hate math :(
Enter intended operation (ONLY [+] [-] [x] [*] [/] [%]): *
Enter another number: Bless me with the leaf of the tree, on it I see, the freedom reign...
Letters and/or symbols are not recognized as numbers. Please try again.
Enter another number: 6
The operation result is: 198
Please only respond with Y or N. Would you like to do another operation? [Y / N]:
n
```

Test case 3: Division by zero

Expected result: Sorry, division by 0 is undefined.

```
Enter a number: 5
Enter intended operation (ONLY [+] [-] [x] [*] [/] [%]): /
Enter another number: 0
Sorry, division by 0 is undefined.
Please only respond with Y or N. Would you like to do another operation? [Y / N]:
```